

LeetCode Notes: 0-1 BFS Approach

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 24, 2026


```

class Solution {
public:
    int minimumObstacles(vector<vector<int>>& grid) {

        int n = grid.size();
        int m = grid[0].size();
        vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};

        // since edge weight are only 0 and 1 we can use deque
        // to store {cost, {row, col}}
        deque<pair<int, pair<int,int>>> dq;
        vector<vector<int>> dist(n, vector<int>(m, INT_MAX));

        dq.push_front({0, {0,0}});
        dist[0][0] = 0;

        while(!dq.empty()){
            int r = dq.front().second.first;
            int c = dq.front().second.second;
            int currcost = dq.front().first;
            dq.pop_front();

            if(currcost > dist[r][c]) continue;
            if(r == n-1 && c == m-1) return currcost;

            for(auto &it : dir){
                int nr = r + it.first;
                int nc = c + it.second;

                if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                    int newcost = currcost;
                    if(grid[nr][nc] == 1){
                        newcost+
+; // kyunki agla block agar blocked he to todna padega
                    }

                    if(newcost < dist[nr][nc]){
                        dist[nr][nc] = newcost;

                        if(newcost == currcost){ // matlab bina tode gaya
                                                // hence as good as currpath ,
so push front
                        dq.push_front({newcost, {nr,nc}});
                    }
                    else{
                        dq.push_back({newcost, {nr,nc}});
                    }
                }
            }
        }
    }
};

```

```

        }
    }
}

return -1;
}
};

```

Reasoning & Notes Breakdown

1. Data Structure Choice (0-1 BFS):

"since edge weight are only 0 and 1 we can use deque to store {cost,{row,col}}"

Using a double-ended queue (deque) allows us to prioritize 0-cost moves over 1-cost moves effectively, mimicking Dijkstra's algorithm but optimized for binary weights.

2. Encountering Obstacles:

"kyunki agla block agar blocked he to todna padega"

If the next cell is `1` (blocked), we must break it. This is represented by incrementing our current path cost (`newcost++`). Because this move costs `1`, it gets pushed to the **back** of our deque to be processed later.

3. Free Path Optimization:

"matlab bina tode gaya hence as good as currpath , so push front"

If the `newcost` equals the `currcost`, it means we moved to an empty cell (`0`). Since this move comes at no extra cost, it's just as good as our current path. By pushing it to the **front** of the deque, we ensure it's processed immediately, keeping the shortest path intact.